Progetto 1 - Big Data

ANAC Query Assistant

Gruppo: Embedding Explorer

Abballe Mirko (566829)
Perniconi Cristian (566835)

Anno Accademico 2025/2026

Il codice utilizzato nel progetto è disponibile su GitHub, accessibile al link seguente:



`https://github.com/Cri-Perni/ANAC_Query_Assistant`

Roma Tre

Indice

1 Introduzione **3**
- 1.1 Contesto del progetto . . . 3
- 1.2 Il problema . . . 3
- 1.3 Obiettivo . . . 3
- 1.4 Dati e dominio applicativo . . . 4
- 1.5 Approccio proposto . . . 4
- 1.6 Contributi principali . . . 4

2 Dati utilizzati **5**
- 2.1 Dati strutturati ANAC . . . 5
- 2.2 Dati normativi . . . 5
- 2.3 Preparazione e criticità dei dati . . . 6

3 Architettura generale **7**
- 3.1 Visione d'insieme . . . 7
- 3.2 Componenti . . . 8
- 3.3 Flusso logico di una richiesta . . . 8
- 3.4 Motivazioni della scelta architetturale . . . 9

4 Implementazione del sistema **10**
- 4.1 Organizzazione dei servizi . . . 10
- 4.2 Layer strutturato . . . 10
  - 4.2.1 Ingestion dei dati su AWS . . . 10
  - 4.2.2 Text-to-SQL . . . 11
- 4.3 Layer semi-strutturato . . . 13
  - 4.3.1 GraphRAG . . . 13
- 4.4 Orchestratore multi-agente . . . 17
- 4.5 Gateway MCP . . . 22
- 4.6 Interfaccia utente Streamlit . . . 25
- 4.7 Gestione delle configurazioni e dei container . . . 27
- 4.8 Considerazioni implementative . . . 28

5 Valutazione sperimentale **29**
- 5.1 Obiettivi della valutazione . . . 29
- 5.2 Dataset di valutazione . . . 29
- 5.3 Configurazione e protocollo sperimentale . . . 30
- 5.4 Metriche . . . 30
- 5.5 Risultati . . . 31
- 5.6 Comportamento del Reflector e del Replanner . . . 31
- 5.7 Analisi degli errori . . . 32
- 5.8 Limiti della valutazione . . . 32
- 5.9 Considerazioni conclusive . . . 33

6 Conclusioni e sviluppi futuri **34**
- 6.1 Evoluzione verso un'architettura cloud AWS . . . 34

Sommario

ANAC Query Assistant è un sistema conversazionale progettato per semplificare l'accesso ai dati sugli appalti pubblici e alla relativa normativa. Attraverso richieste in linguaggio naturale, il sistema integra dati strutturati interrogati con Amazon Athena e contenuti normativi recuperati mediante Neo4j GraphRAG.
Il progetto adotta un'architettura a microservizi e un orchestratore multi-agente basato su LangGraph, responsabile della pianificazione, dell'esecuzione degli strumenti, della valutazione dei risultati e della sintesi finale. Un Gateway MCP centralizza e uniforma l'accesso ai servizi di dominio.
Il sistema si distingue per:

- Interfaccia conversazionale realizzata con Streamlit
- Generazione automatica di query SQL mediante il servizio Text-to-SQL
- Recupero semantico e temporale della normativa tramite Neo4j GraphRAG
- Orchestrazione multi-agente con valutazione, feedback e ripianificazione
- Guardrail di dominio e tracciamento delle operazioni eseguite

1 Introduzione

1.1 Contesto del progetto

Negli ultimi anni i dati pubblici sugli appalti hanno assunto un ruolo sempre più rilevante nelle attività di monitoraggio, analisi e controllo della spesa pubblica. In Italia, la Banca Dati Nazionale dei Contratti Pubblici e gli open data messi a disposizione da ANAC raccolgono informazioni su gare, CIG, importi, stazioni appaltanti, aggiudicazioni e operatori economici coinvolti nelle procedure di affidamento.
Il dominio degli appalti pubblici è particolarmente complesso perché combina grandi volumi di dati operativi con un articolato quadro normativo. Le informazioni quantitative descrivono le procedure, gli importi e i soggetti coinvolti; i testi legislativi ne definiscono invece il significato giuridico, attraverso articoli, commi, allegati, soglie economiche e rimandi tra disposizioni.
Questa coesistenza rende il dominio adatto a un progetto Big Data: il **volume** richiede strumenti scalabili di interrogazione, la **varietà** impone tecniche diverse per trattare tabelle e testi normativi, mentre la **veridicità** è essenziale per produrre risposte fondate su fonti verificabili.

1.2 Il problema

Il problema affrontato riguarda la difficoltà di interrogare in modo congiunto dati strutturati e documentazione normativa. I dataset ANAC permettono di calcolare fenomeni quantitativi, come il numero di gare o la distribuzione degli importi, ma non chiariscono da soli il significato giuridico delle procedure. Al contrario, i testi normativi forniscono il contesto interpretativo, ma non permettono di rispondere direttamente a domande statistiche sui dati.
Senza un sistema integrato, l'utente dovrebbe tradurre manualmente la richiesta in query SQL, conoscere lo schema del data lake, individuare le colonne corrette e collegare i risultati alla normativa applicabile. Questo processo è oneroso, soggetto a errori e poco accessibile a utenti privi di competenze tecniche o giuridiche avanzate.
Il progetto nasce quindi dall'esigenza di ridurre la distanza tra analisi dei dati e interpretazione normativa, consentendo all'utente di porre domande in linguaggio naturale e ricevere risposte comprensibili, motivate e tracciabili.

1.3 Obiettivo

L'obiettivo del lavoro è progettare e realizzare un **assistente intelligente** per l'analisi degli appalti pubblici italiani. Il sistema deve interpretare domande in linguaggio naturale, selezionare gli strumenti più adatti e produrre risposte basate su dati verificabili.
Il progetto si colloca nel tema del ***Data-Engineer AI Agent***, poiché combina accesso al dato, traduzione automatica delle richieste, recupero di conoscenza e orchestrazione agentica. L'assistente non si limita a generare una query o a recuperare documenti, ma coordina componenti specializzate e decide se utilizzare il modulo strutturato, il modulo normativo o entrambi.

1.4 Dati e dominio applicativo

Il dominio applicativo scelto è quello degli appalti pubblici italiani. La parte strutturata riguarda i dati ANAC relativi alle gare e ai CIG pubblicati tra il 2007 e il 2025; la parte normativa comprende i principali codici dei contratti pubblici, in particolare il **D.Lgs. 163/2006**, il **D.Lgs. 50/2016** e il **D.Lgs. 36/2023**.
Queste fonti permettono di coprire due esigenze complementari: analizzare i fenomeni osservabili nei dati e contestualizzarli rispetto alla disciplina normativa. La descrizione dettagliata dei dataset e delle trasformazioni applicate è riportata nel capitolo successivo.

1.5 Approccio proposto

L'approccio proposto consiste nella realizzazione di un **assistente multi-agente** capace di integrare dati strutturati sugli appalti pubblici e conoscenza normativa. Il sistema riceve domande in linguaggio naturale e seleziona il percorso di elaborazione più adatto: interrogazione dei dati ANAC, recupero di informazioni normative oppure combinazione di entrambi.
La soluzione si basa su componenti specializzati: un modulo Text-to-SQL per i dati strutturati, un modulo GraphRAG per il recupero normativo e un orchestratore che coordina i diversi strumenti. I dettagli relativi alla loro organizzazione e al ruolo di LangGraph sono descritti nel capitolo dedicato all'architettura generale.

1.6 Contributi principali

I principali contributi del progetto sono:

- realizzazione di un sistema integrato per interrogare dati strutturati e testi normativi sugli appalti pubblici;
- implementazione di un modulo Text-to-SQL con validazione delle query, controllo read-only e utilizzo del catalogo semantico del data lake;
- costruzione di un knowledge graph normativo in Neo4j, arricchito con embedding, topic, concetti semantici e relazioni tra disposizioni;
- sviluppo di un motore GraphRAG che combina recupero da grafo, ricerca vettoriale e reranking;
- progettazione di un orchestratore agentico basato su LangGraph;
- sviluppo di un'interfaccia Streamlit per l'interazione con il sistema.

2 Dati utilizzati

Il sistema utilizza due insiemi informativi principali: i dati strutturati ANAC, destinati alle interrogazioni quantitative, e i documenti normativi, trasformati in una base di conoscenza per il recupero semantico e relazionale. Questa scelta riflette la natura del dominio: una domanda sugli appalti può richiedere un conteggio, un filtro temporale, un importo, oppure il richiamo a una disposizione del codice dei contratti pubblici.

2.1 Dati strutturati ANAC

La componente strutturata è costituita principalmente dai dati relativi ai CIG e alle gare pubbliche. Il CIG, Codice Identificativo Gara, rappresenta l'elemento centrale del dataset perché consente di collegare informazioni su gara, importi, stazione appaltante, aggiudicazioni e operatori economici.
Nel progetto sono stati considerati i file annuali dei CIG **dal 2007 al 2025**. La raccolta comprende **19 file CSV**, uno per ciascun anno, per un totale di **11.046.176 record** e una dimensione complessiva di circa **6,83 GB**. Il volume dei dati giustifica l'uso di strumenti di interrogazione scalabile, in particolare Amazon Athena e AWS Glue.

Fonte	**Periodo**	**Record**	**Ruolo nel progetto**
File `CIG_YYYY.csv`	2007–2025	11.046.176	Base principale per l'analisi delle gare e dei CIG.
`cig.csv`	Dataset aggregato	642.593	Estratto utilizzato per analisi preliminari e verifiche sullo schema.
`smartcig.csv`	Dataset aggregato	864.784	Fonte relativa a procedure semplificate e affidamenti di minore complessità.

Tabella 1: Principali dataset strutturati considerati nel progetto.

Le informazioni disponibili includono attributi temporali, identificativi di gara, oggetto del contratto, importi, stazioni appaltanti, classificazioni CPV, dati territoriali ed esiti delle procedure. Il campo temporale più rilevante per le interrogazioni è `anno_pubblicazione`, utilizzato per filtrare le gare rispetto all'anno richiesto dall'utente.
Nel sistema finale, AWS Glue mantiene il catalogo dei metadati e Amazon Athena esegue le query sul data lake. Il modulo Text-to-SQL utilizza inoltre un catalogo semantico interno per distinguere le tabelle dedicate ai CIG, alle aggiudicazioni e agli aggiudicatari, riducendo il rischio di generare query non coerenti con lo schema.

2.2 Dati normativi

La componente normativa è formata dai testi legislativi che regolano il settore degli appalti pubblici. Sono stati considerati tre decreti legislativi, scelti perché rappresentano le principali fasi di evoluzione del codice dei contratti pubblici:

- **D.Lgs. 163/2006**, relativo al precedente codice dei contratti pubblici;
- **D.Lgs. 50/2016**, che ha sostituito il codice del 2006;
- **D.Lgs. 36/2023**, corrispondente al nuovo codice dei contratti pubblici.

I documenti vengono elaborati da una pipeline di ingestion che estrae il testo dai PDF, identifica leggi, articoli, commi e allegati, e produce file CSV destinati al caricamento in Neo4j. Le unità testuali più granulari vengono arricchite con embedding, topic e concetti semantici, così da supportare sia il recupero tramite grafo sia la ricerca per similarità.

Elemento	**Quantità**	**Descrizione**
Leggi	3	Decreti legislativi indicizzati nella base normativa.
Articoli	706	Articoli estratti dai tre decreti.
Allegati	102	Allegati normativi associati ai decreti.
Commi	3.939	Unità testuali usate per il recupero normativo.
Embedding dei commi	3.939	Vettori semantici associati ai commi/clausole.
Topic	54	Categorie tematiche utilizzate per arricchire il grafo.
Concetti semantici	54	Concetti di dominio collegati ad articoli e clausole.
Relazioni articolo–comma	3.939	Collegamenti tra articoli e commi/clausole.
Relazioni comma–topic	4.486	Associazioni tra clausole e categorie tematiche.
Relazioni comma–concetto	2.581	Associazioni tra clausole e concetti semantici.

Tabella 2: Principali elementi prodotti dalla pipeline di ingestion normativa.

La rappresentazione a grafo consente di modellare relazioni esplicite tra leggi, articoli, commi, allegati e concetti di dominio. In questo modo il GraphRAG può recuperare non solo passaggi testuali simili alla domanda, ma anche il contesto normativo collegato.

2.3 Preparazione e criticità dei dati

La preparazione segue due percorsi distinti. Per il layer strutturato, i CSV vengono raccolti, normalizzati e resi disponibili nel data lake, con attenzione alla coerenza dello schema, ai valori mancanti, agli importi e ai campi temporali. Per il layer normativo, i PDF vengono trasformati in un knowledge graph tramite estrazione del testo, segmentazione in articoli e commi, generazione degli embedding e caricamento in Neo4j.
L'uso di dati reali introduce alcune criticità. Nei dataset ANAC lo schema può variare nel tempo e alcuni campi possono essere incompleti o valorizzati in modo non uniforme. Nei testi normativi, invece, la presenza di rimandi, allegati e modifiche legislative rende complesso individuare il contesto corretto. Per questo il sistema combina cataloghi semantici, validazione delle query, relazioni esplicite nel grafo e recupero vettoriale.
Non sempre esiste un collegamento diretto tra un record di gara e una specifica disposizione legislativa. In molti casi l'integrazione avviene a livello interpretativo: il sistema calcola o recupera un risultato dai dati ANAC e lo contestualizza rispetto alla normativa applicabile. Questo è uno dei motivi per cui l'orchestratore svolge un ruolo centrale nel progetto.

3 Architettura generale

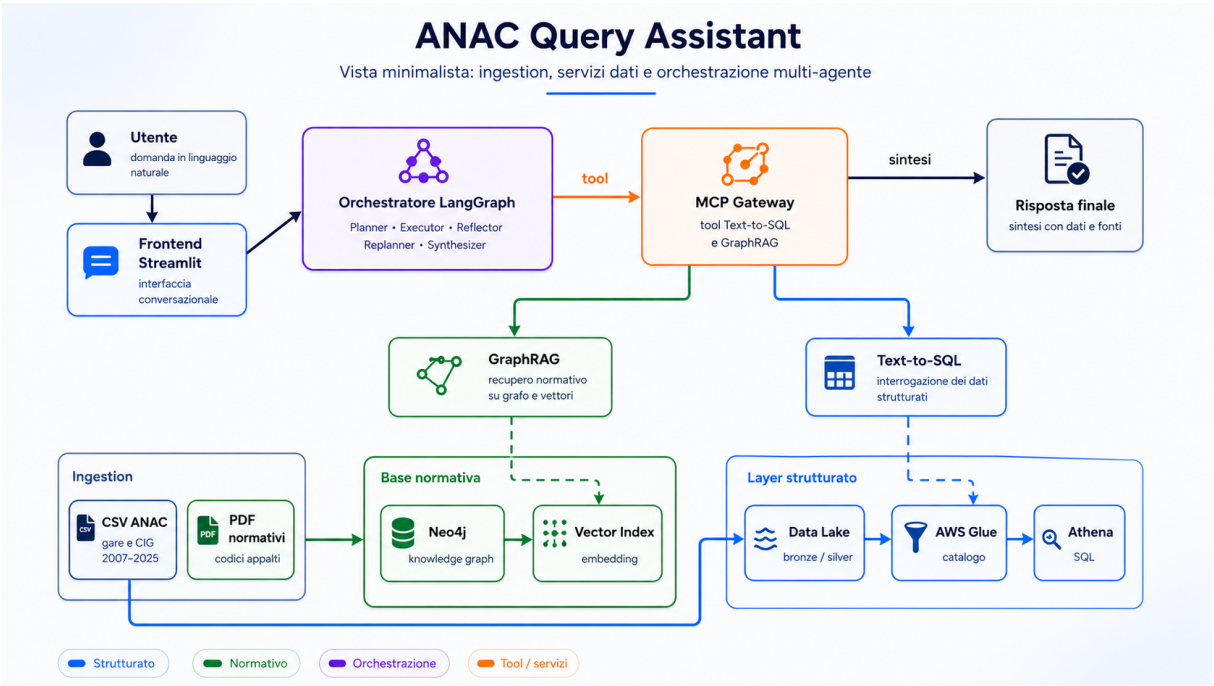


Figura 1: Architettura sintetica di ANAC Query Assistant

L'architettura di ANAC Agent è stata progettata per integrare due forme di conoscenza diverse: i dati strutturati sugli appalti pubblici e la documentazione normativa sui contratti pubblici. Il capitolo non entra nei dettagli implementativi dei singoli moduli, che saranno descritti nel capitolo successivo, ma presenta la struttura logica del sistema e il ruolo dei principali componenti.
La scelta architetturale principale consiste nel separare l'interfaccia utente, l'orchestrazione, i servizi applicativi e i livelli di persistenza. In questo modo ogni componente ha una responsabilità precisa e può essere evoluto senza modificare l'intero sistema.

3.1 Visione d'insieme

Da un punto di vista logico, il sistema può essere suddiviso in quattro livelli:

- **livello di interfaccia**, attraverso il quale l'utente formula domande in linguaggio naturale;
- **livello di orchestrazione**, che interpreta la richiesta e decide quali strumenti utilizzare;
- **livello dei servizi applicativi**, composto dai moduli specializzati per dati strutturati e conoscenza normativa;
- **livello dati**, formato dal data lake ANAC e dal knowledge graph normativo.

La Figura 1 mostra una vista sintetica dell'architettura. Il frontend non interroga direttamente le sorgenti dati, ma invia la richiesta all'orchestratore. Quest'ultimo seleziona il percorso più adatto: interrogazione strutturata, recupero normativo oppure combinazione dei due.

3.2 Componenti

La Tabella 3 riassume i componenti principali. In questa sezione il loro ruolo è descritto a livello architetturale; le scelte implementative sono approfondite nel capitolo successivo.

Componente	**Tecnologia**	**Ruolo architetturale**
Frontend	Streamlit	Punto di accesso conversazionale per l'utente.
Orchestratore	FastAPI, LangGraph	Coordina il processo di risposta e decide quali strumenti invocare.
MCP Gateway	FastMCP	Espone i servizi applicativi come tool standardizzati.
Text-to-SQL	FastAPI, LLM, Athena	Permette l'interrogazione dei dati strutturati ANAC.
GraphRAG	FastAPI, Neo4j, embedding	Permette il recupero di conoscenza normativa dal grafo.
Data lake	AWS Glue, Amazon Athena	Conserva e rende interrogabili i dati strutturati.
Knowledge graph	Neo4j	Rappresenta leggi, articoli, commi, allegati e relazioni normative.

Tabella 3: Componenti principali dell'architettura di ANAC Agent.

Questa suddivisione consente di mantenere separati i percorsi di accesso alla conoscenza. Il modulo Text-to-SQL è responsabile delle domande quantitative, mentre il GraphRAG è responsabile delle domande normative. L'orchestratore rappresenta il punto di composizione tra questi percorsi.

3.3 Flusso logico di una richiesta

Il flusso inizia quando l'utente invia una domanda tramite l'interfaccia Streamlit. La richiesta viene inoltrata all'orchestratore, che analizza il contenuto della domanda e costruisce un piano di esecuzione. Se la domanda riguarda dati numerici, statistiche o filtri sui CIG, viene attivato il percorso strutturato. Se riguarda articoli, commi, allegati, soglie o disposizioni normative, viene attivato il percorso GraphRAG.
Alcune domande richiedono entrambi i percorsi. Ad esempio, una richiesta sulle gare sopra soglia può richiedere prima il recupero della soglia normativa e poi una query sui dati ANAC. In questo caso l'architettura permette di combinare più strumenti all'interno di una stessa risposta.
Il risultato finale viene prodotto dall'orchestratore, che sintetizza le informazioni recuperate e restituisce all'utente una risposta comprensibile. Il sistema può inoltre mostrare gli eventi principali dell'elaborazione, rendendo più trasparente il percorso seguito dall'agente.

3.4 Motivazioni della scelta architetturale

La scelta di un'architettura modulare deriva da tre esigenze principali. La prima è la **separazione delle responsabilità**: ogni servizio gestisce una parte specifica del problema. La seconda è la **manutenibilità**: i moduli possono essere aggiornati o sostituiti con impatto limitato sugli altri componenti. La terza è l'**estensibilità**: nuovi tool, nuove fonti dati o nuove strategie di recupero possono essere integrati mantenendo invariato il ruolo generale dell'orchestratore.
Questa impostazione è coerente con il dominio degli appalti pubblici, in cui una richiesta può essere quantitativa, normativa o ibrida. L'architettura consente quindi di mantenere distinti i diversi tipi di conoscenza, integrandoli solo quando la domanda dell'utente lo richiede.

4 Implementazione del sistema

Dopo aver descritto l'architettura generale, questo capitolo approfondisce le principali **scelte implementative** adottate per realizzare ANAC Agent. L'attenzione non è posta nuovamente sulla struttura complessiva del sistema, ma sul funzionamento interno dei moduli applicativi e sulle logiche che permettono di trasformare una domanda in linguaggio naturale in una risposta basata su dati e fonti normative.

4.1 Organizzazione dei servizi

Il sistema è stato implementato come insieme di servizi indipendenti. Ogni modulo espone una responsabilità precisa: il frontend gestisce l'interazione con l'utente, l'orchestratore coordina il processo di risposta, il gateway MCP rende disponibili gli strumenti invocabili dall'agente, mentre i servizi Text-to-SQL e GraphRAG si occupano rispettivamente del layer strutturato e del layer normativo.
Questa organizzazione consente di sviluppare e testare i componenti in modo separato.

4.2 Layer strutturato

4.2.1 Ingestion dei dati su AWS



Figura 2: Pipeline di ingestion del layer strutturato su AWS

La fase di ingestion ha l'obiettivo di predisporre un **data lake interrogabile**, separando i file originali dai dataset trasformati. I dati ANAC sono stati caricati in un **bucket Amazon S3** denominato `anac-data-lake`, organizzato secondo una distinzione tra livello *bronze* e livello *silver*. Il livello *bronze* conserva i file di partenza, come il dataset `CIG.csv`, mantenendo una rappresentazione vicina alla sorgente originale.

Il livello *silver*, invece, contiene i dati trasformati e organizzati per dominio informativo. In questa area sono presenti cartelle come `silver_cig/`, `silver_aggiudicazioni/`, `silver_aggiudicatari/` e `silver_stazioni/`. I file del livello silver sono memorizzati in formato **Parquet**: questa scelta è coerente con l'utilizzo di Athena, poiché il formato colonnare permette di leggere soltanto le colonne necessarie alla query e riduce il volume di dati scansionato rispetto ai CSV.
Come mostrato in Figura 2, il processo parte dai dataset CSV e produce tabelle catalogate. Dopo il caricamento su S3, i dati vengono puliti, tipizzati e suddivisi in file Parquet all'interno delle cartelle silver. AWS Glue Data Catalog viene poi utilizzato per registrare le tabelle, i metadati e lo schema delle colonne nel database `anac_db`. Ad esempio, la tabella `silver_cig` punta alla posizione `s3://anac-data-lake/silver_cig/` e utilizza una serializzazione Parquet compatibile con Hive.
Athena interroga direttamente i file presenti nel bucket S3 utilizzando lo schema catalogato da Glue. Non è quindi necessario importare i dati in un database relazionale tradizionale: il data lake rimane la sorgente fisica, mentre Glue fornisce il catalogo logico e Athena esegue le query SQL. La cartella `athena-results/` viene usata come area di output per i risultati delle interrogazioni.
Questa impostazione è il prerequisito del modulo Text-to-SQL. Il modello non genera query su file locali, ma su tabelle effettivamente presenti nel catalogo Glue e interrogabili tramite Athena. In questo modo la generazione SQL può essere vincolata allo schema reale del progetto e l'orchestratore riceve risultati già coerenti con il layer strutturato.

4.2.2 Text-to-SQL

Il modulo Text-to-SQL è implementato come servizio FastAPI e rappresenta il punto di accesso al layer strutturato del progetto. Il suo compito non è soltanto generare una query SQL a partire da una domanda in linguaggio naturale, ma costruire una query compatibile con Amazon Athena, coerente con lo schema reale del data lake e sicura dal punto di vista operativo.
Il servizio espone principalmente gli endpoint `/api/nl2sql`, `/api/query`, `/api/tables` e `/api/presets`. Il primo traduce una richiesta testuale in SQL, il secondo esegue una query validata su Athena, il terzo restituisce le tabelle disponibili nel catalogo e il quarto permette di consultare query predefinite. Questa separazione consente di distinguere chiaramente la fase di generazione della query dalla fase di esecuzione.

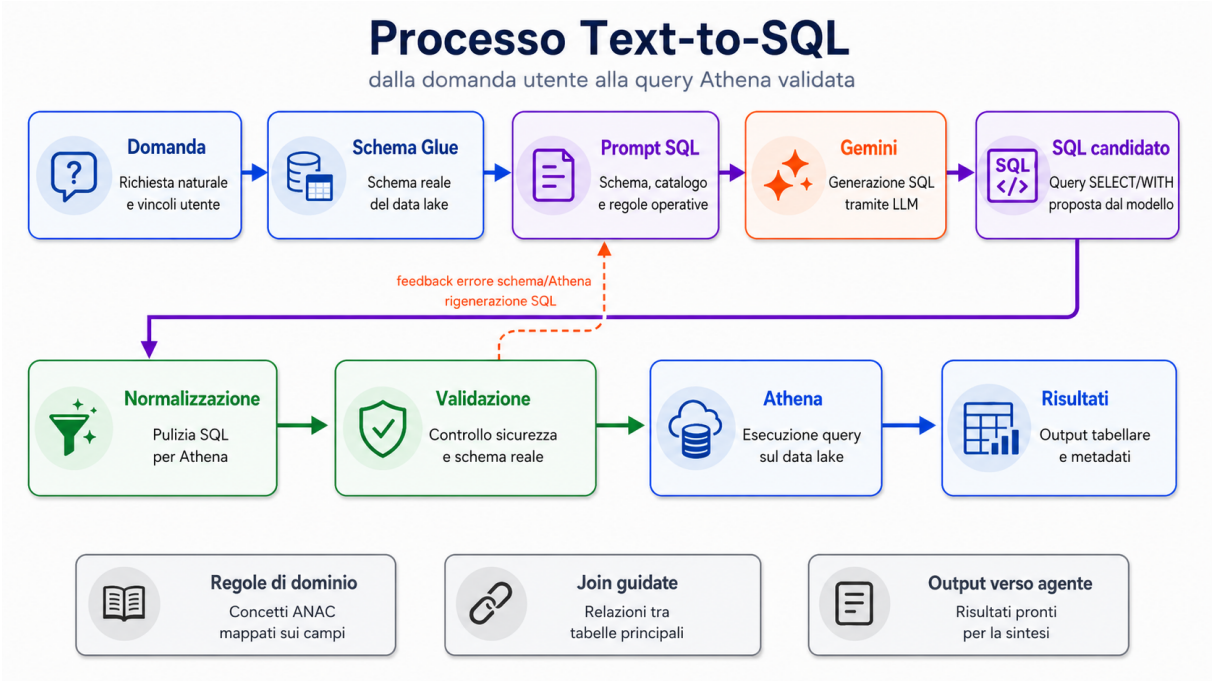


Figura 3: Pipeline interna del modulo Text-to-SQL

Come mostrato in Figura 3, il processo inizia dalla richiesta dell'utente e dalla costruzione del contesto da fornire al modello linguistico. Prima della generazione SQL, il servizio recupera lo schema delle tabelle da AWS Glue Data Catalog. Vengono considerate in particolare le tabelle del data lake con prefisso `silver`, poiché rappresentano i livelli già organizzati per l'analisi. Lo schema viene mantenuto in cache per ridurre chiamate ripetute a Glue e viene arricchito con alcuni valori campione per colonne testuali rilevanti, così da aiutare il modello a usare valori realmente presenti nei dati.

A questo schema tecnico viene aggiunto un catalogo semantico interno. Il catalogo descrive il significato delle principali tabelle e colonne del dominio ANAC. Ad esempio, `silver_cig` viene indicata come tabella principale per gare, CIG, lotti, importi e anni di pubblicazione; `silver_aggiudicazioni` viene utilizzata quando la domanda riguarda esiti o importi di aggiudicazione; `silver_aggiudicatari` viene usata per operatori economici, vincitori e fornitori. Il catalogo contiene anche indicazioni sulle join consigliate, come il collegamento tramite `cig` tra CIG e aggiudicazioni, o tramite `id_aggiudicazione` tra aggiudicazioni e aggiudicatari.

Il prompt inviato al modello include quindi tre elementi: lo schema reale Athena, il catalogo semantico e un insieme di regole operative. Tra queste regole ci sono vincoli sul dialetto SQL Presto/Trino, istruzioni sull'uso di `anno_pubblicazione` per i filtri temporali, `importo_complessivo_gara` per i confronti economici, `oggetto_principale_contratto` per distinguere lavori, servizi e forniture, e `settore` per distinguere settori ordinari e speciali.

Una parte specifica riguarda i filtri geografici. Poiché nel dataset non è sempre disponibile una colonna regionale diretta, il servizio riconosce eventuali regioni italiane presenti nella domanda e costruisce un hint basato sulle province corrispondenti. Ad esempio, una richiesta sugli appalti nel Lazio viene trasformata in un vincolo sulle province di Roma, Frosinone, Latina, Rieti e Viterbo. Questo evita che il modello generi condizioni su colonne inesistenti come `regione`.

Dopo la generazione da parte del modello, la query non viene eseguita immediatamente. Prima viene ripulita da eventuali blocchi Markdown, normalizzata rispetto alle convenzioni di Athena e corretta in alcuni casi noti, come l'uso di apostrofi nelle stringhe SQL o forme non supportate da Presto, ad esempio `COUNT(DISTINCT ...)` all'interno di funzioni finestra.
La validazione avviene su due livelli. Il primo livello verifica che la query sia in sola lettura: sono ammesse soltanto interrogazioni che iniziano con `SELECT` o `WITH`, mentre vengono bloccate istruzioni che alterano il contenuto della tabella, come `INSERT`, `UPDATE`, etc. Il secondo livello confronta la query con lo schema reale: vengono controllate le tabelle citate nelle clausole `FROM` e `JOIN` e, quando possibile, anche i riferimenti qualificati del tipo `alias.colonna`. Se una colonna non è presente, il sistema produce un errore con possibili colonne simili.
L'esecuzione avviene su Amazon Athena in modo asincrono. Il servizio avvia la query, attende il completamento, recupera le righe tramite paginazione e restituisce colonne, risultati, tempo di esecuzione e informazione sull'eventuale troncamento.
Il modulo prevede infine meccanismi di robustezza. Se la query generata non supera la validazione rispetto allo schema, il messaggio di errore viene reinserito nel prompt e il modello può tentare una nuova generazione. Inoltre sono configurati modelli di fallback, così da non dipendere da un solo modello nel caso di indisponibilità temporanea. Gli errori più comuni restituiti da Athena, come colonne inesistenti, tabelle mancanti o problemi di sintassi, vengono trasformati in messaggi controllati e interpretabili dall'orchestratore.

4.3 Layer semi-strutturato

4.3.1 GraphRAG

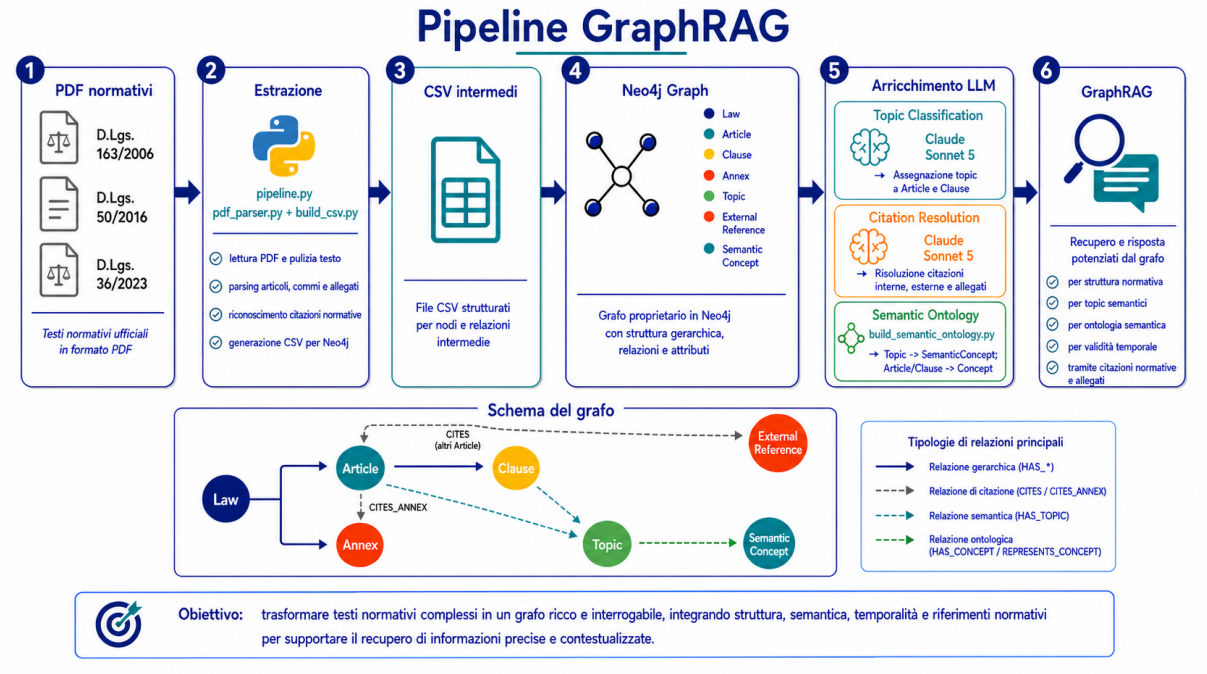


Figura 4: Pipeline GraphRAG

Il modulo GraphRAG costituisce la componente dedicata alla conoscenza normativa non strutturata. Il suo obiettivo è trasformare i testi dei decreti legislativi in una base interrogabile che preservi sia la struttura formale della norma sia le relazioni semantiche e citazionali tra le disposizioni. La pipeline è composta da una fase *offline*, dedicata alla costruzione e all'arricchimento del grafo, e da una fase *online*, eseguita durante l'interazione con l'utente.

Acquisizione e parsing dei documenti normativi. I documenti sorgente sono costituiti dai PDF dei principali Codici dei contratti pubblici analizzati nel progetto: D.Lgs. 163/2006, D.Lgs. 50/2016 e D.Lgs. 36/2023. Il processo è coordinato dal file `pipeline.py`, mentre l'estrazione del testo è gestita da `pdf_parser.py`.
Per ciascun PDF viene dapprima ricavato il testo pagina per pagina mediante la libreria `pypdf`. Durante questa fase vengono inseriti marcatori di pagina interni, utili per mantenere la provenienza del contenuto anche dopo la segmentazione. Il testo viene poi normalizzato tramite espressioni regolari: vengono rimossi artefatti tipografici, intestazioni e piè di pagina ripetuti, spaziature anomale e altri elementi che potrebbero compromettere il riconoscimento della struttura normativa.
Dal nome del file e dal suo contenuto vengono inoltre derivati i metadati del decreto: identificativo della legge, anno, titolo, intervallo di validità, stato della norma e versione della fonte. Tale informazione permette di distinguere, ad esempio, il D.Lgs. 163/2006 e il D.Lgs. 50/2016, modellati come non più vigenti, dal D.Lgs. 36/2023, considerato vigente alla data di realizzazione del progetto.

Costruzione della struttura giuridica. Il modulo `build_csv.py`, con il supporto delle regole definite in `regex_patterns.py`, riconosce gli elementi della struttura normativa. Il testo viene quindi suddiviso in articoli, commi e allegati. Questa scelta consente di mantenere unità informative giuridicamente significative e direttamente citabili nella risposta finale.
Per ogni elemento vengono creati identificativi deterministici. Un articolo assume, ad esempio, una forma simile a `d_lgs_36_2023:art:50`, mentre un comma eredita l'identificativo dell'articolo padre. La segmentazione produce i nodi principali:

- `Law`, corrispondente al decreto legislativo;
- `Article`, contenente numero, titolo, testo, ordine e pagina di origine;
- `Clause`, corrispondente al comma dell'articolo;
- `Annex`, per gli allegati normativi;
- `Topic` e `SemanticConcept`, utilizzati nell'arricchimento semantico;
- `ExternalReference`, per i riferimenti a fonti esterne al corpus.

La **struttura gerarchica** viene esportata in **file CSV intermedi**, insieme alle **relazioni** `HAS_ARTICLE`, `HAS_CLAUSE` e `HAS_ANNEX`. Vengono inoltre generate le relazioni `NEXT_ARTICLE` e `NEXT_CLAUSE`, che mantengono l'ordine originario delle disposizioni e possono essere sfruttate nelle esplorazioni locali del grafo.

Risoluzione delle citazioni e modellazione temporale. Una fase successiva analizza il testo di articoli e commi per individuare i rinvii normativi. Le **regole deterministiche** riconoscono riferimenti quali "articolo 50", "articoli 16 e 17" oppure richiami a specifici commi. Quando il riferimento può essere risolto all'interno del corpus, vengono generate relazioni come `CITES_ARTICLE` e `CITES_CLAUSE`; i riferimenti a fonti non presenti nel grafo sono invece rappresentati mediante nodi `ExternalReference` e relazioni `CITES_EXTERNAL`.
Le citazioni che non possono essere risolte con sufficiente sicurezza vengono memorizzate in `unresolved_citations.csv` e analizzate dal classificatore LLM. Il modello può distinguere tra riferimenti interni allo stesso decreto, riferimenti a un altro decreto del corpus, riferimenti esterni e casi non risolvibili. In questo modo il modello generativo non sostituisce le regole sintattiche, ma interviene soltanto nei casi ambigui.
Il grafo include anche il susseguirsi temporale dei Codici dei contratti pubblici attraverso relazioni di sostituzione tra D.Lgs. 163/2006, D.Lgs. 50/2016 e D.Lgs. 36/2023. I metadati di validità sono utilizzati in **fase di recupero** per privilegiare la normativa applicabile all'anno richiesto dall'utente.

Arricchimento semantico ed embedding. L'arricchimento semantico avviene su due livelli. Inizialmente, articoli e commi vengono associati a un insieme controllato di topic, quali affidamento diretto, soglie, RUP, subappalto, concessioni e criteri di aggiudicazione. Il primo collegamento può essere ottenuto mediante parole chiave; il classificatore LLM produce poi relazioni più contestuali, memorizzando anche confidenza, motivazione e ruolo del topic.
I topic vengono successivamente trasformati in concetti semantici tipizzati, quali procedure, requisiti, soglie monetarie, attori pubblici, attori privati e principi giuridici. Le relazioni `HAS_TOPIC` e `REPRESENTS_CONCEPT` rendono possibile collegare disposizioni formalmente distanti ma riferite al medesimo istituto giuridico.
Per supportare la **ricerca semantica**, viene generato un **embedding** per ciascun nodo `Clause`. Il processo utilizza il modello `gemini-embedding-2`, memorizza gli embedding in `clause_embeddings.csv`. Durante il caricamento, Neo4j crea l'**indice vettoriale** `clause_embeddings_idx`, configurato con **similarità coseno**.

Caricamento in Neo4j. Il modulo `neo4j_uploader.py` importa nodi, relazioni e vettori nel database Neo4j. Vengono definite chiavi univoche per gli identificativi dei nodi, così da evitare duplicazioni e consentire aggiornamenti incrementali. Il caricamento separa logicamente la creazione dei nodi dalla creazione delle relazioni, preservando la coerenza della struttura gerarchica, citazionale e semantica.
Il risultato è un **knowledge graph** in cui un comma può essere raggiunto attraverso percorsi differenti: dalla legge di appartenenza, dall'articolo padre, da un rinvio normativo, da un topic o da un concetto semantico. Questa ridondanza controllata non replica il testo, ma amplia le modalità con cui l'informazione può essere recuperata.

Recupero e generazione della risposta.

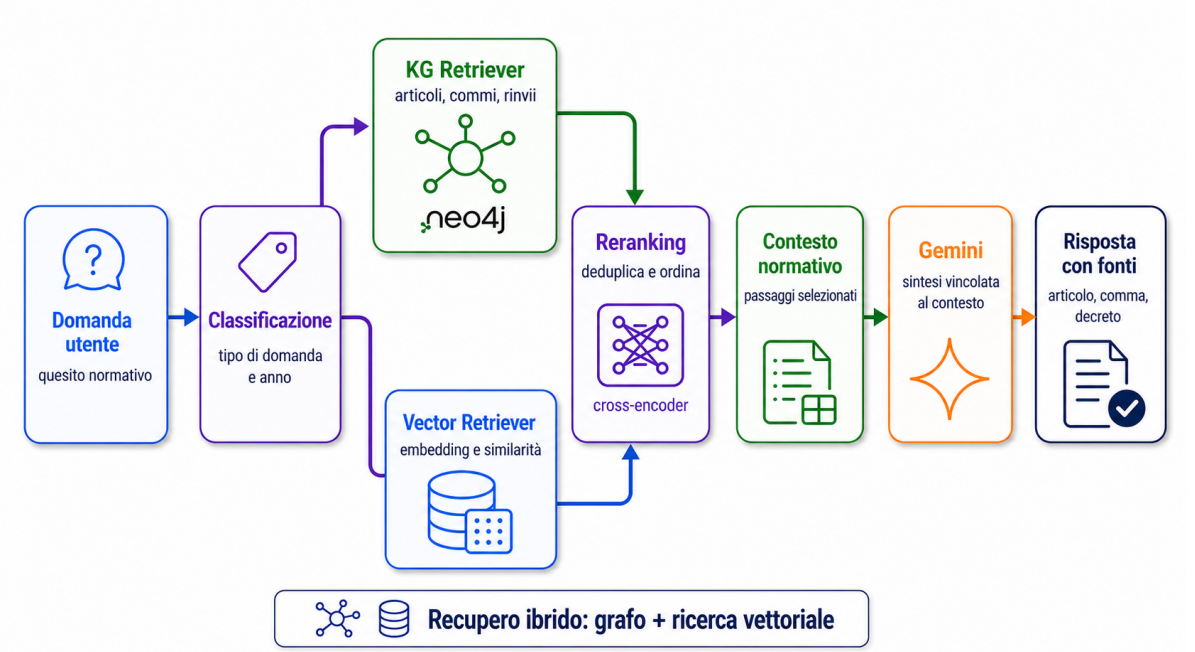


Figura 5: Processo di recupero ibrido e generazione della risposta

In fase di interrogazione, `rag_engine.py` esegue una pipeline articolata in quattro stadi: classificazione della domanda, recupero, reranking e generazione della risposta. Dalla richiesta dell'utente viene anzitutto estratto l'eventuale anno di riferimento. Il classificatore combina regole basate su espressioni regolari con un fallback a Gemini per i casi ambigui, individuando in particolare la presenza di riferimenti espliciti ad articoli oppure la natura generica del quesito.

Il recupero combina due strategie eseguite in parallelo. Quando la domanda cita uno o più articoli, il *Knowledge Graph Retriever* utilizza tali articoli come nodi iniziali della traversata e recupera i commi, i rinvii normativi e i concetti semantici collegati. In assenza di riferimenti espliciti, il suo contributo può essere vuoto. Il *Vector Retriever*, invece, trasforma la domanda in embedding ed effettua una ricerca di similarità sui commi indicizzati. Per le richieste più articolate, il sistema può inoltre generare riformulazioni giuridiche della domanda e calcolarne una rappresentazione vettoriale media.

I passaggi provenienti dalle due sorgenti vengono normalizzati, deduplicati e ordinati mediante il modello `cross-encoder/mmarco-mMiniLMv2-L12-H384-v1`. Il reranker valuta congiuntamente la domanda dell'utente e il testo di ciascun passaggio, assegnando un punteggio di pertinenza. I contenuti ottenuti tramite traversata del grafo ricevono una lieve priorità, poiché mantengono un collegamento esplicito con la struttura normativa e con i rinvii individuati. Il contesto finale viene inoltre limitato a un budget massimo di token, evitando che informazioni marginali riducano la qualità della sintesi.

Infine, il modello generativo riceve esclusivamente i passaggi normativi selezionati. Il prompt impone di non utilizzare conoscenza esterna e di citare articolo, comma e decreto di provenienza. La risposta prodotta è quindi accompagnata dalle fonti utilizzate, rendendo il processo maggiormente verificabile e adatto a un dominio normativo.

4.4 Orchestratore multi-agente

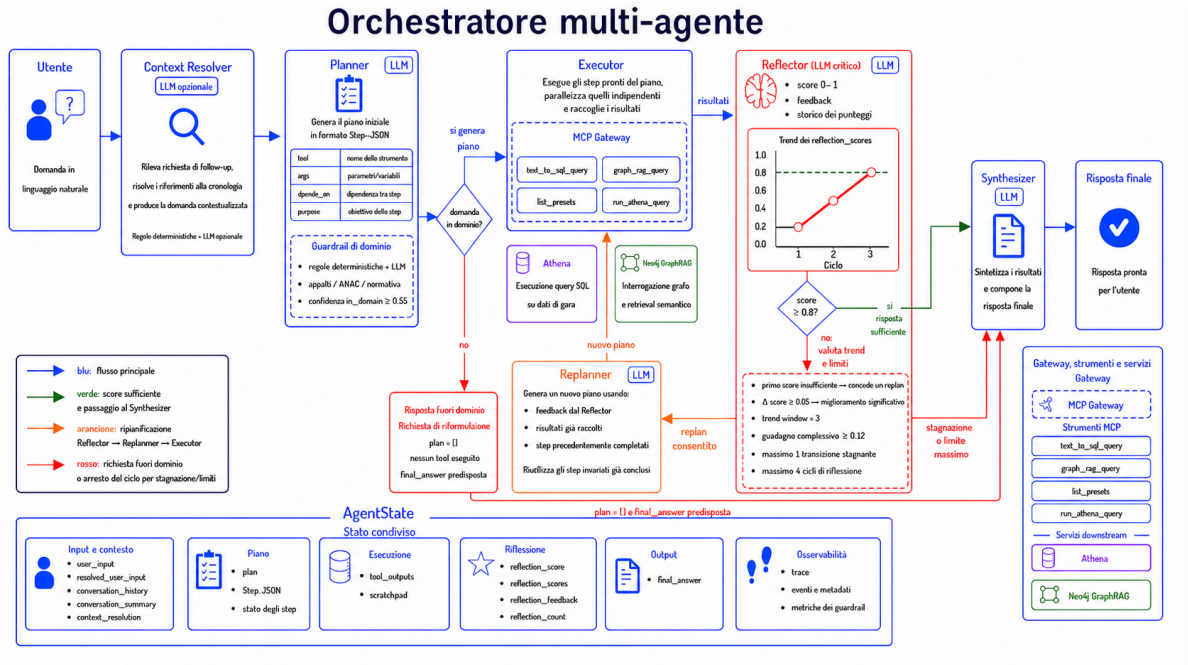


Figura 6: Architettura dell'orchestratore multi-agente

L'orchestratore multi-agente rappresenta il livello di coordinamento dell'applicazione. Il suo compito non è eseguire direttamente interrogazioni SQL o ricerche normative, ma trasformare la richiesta dell'utente in un processo controllato composto da più fasi: comprensione del contesto, pianificazione, esecuzione degli strumenti, valutazione critica, eventuale ripianificazione e sintesi finale. L'implementazione utilizza `LangGraph`, che consente di modellare l'intero processo come una macchina a stati, in cui ogni nodo svolge un ruolo specifico e il passaggio da uno stato al successivo è regolato da archi fissi o condizionali.
L'espressione *multi-agente* non indica la presenza di processi completamente indipendenti, ma una separazione esplicita dei ruoli decisionali all'interno dello stesso workflow. Ogni agente riceve una porzione mirata dello stato condiviso, esegue una responsabilità circoscritta e aggiorna lo stato prima di passare il controllo al nodo successivo. Gli agenti implementati sono ***Context Resolver***, ***Planner***, ***Executor***, ***Reflector***, ***Replanner*** e ***Synthesizer***.

Stato condiviso e struttura del workflow. L'intero workflow è governato da una struttura condivisa denominata `AgentState`. Essa rappresenta la memoria operativa dell'orchestratore durante una singola richiesta e contiene tutti gli elementi necessari per coordinare i nodi del grafo. In particolare, lo stato conserva la domanda originale dell'utente, la cronologia conversazionale, l'eventuale domanda risolta rispetto al contesto, il piano di esecuzione, gli output degli strumenti, lo `scratchpad` con le evidenze raccolte, il punteggio dell'ultima reflection, la lista degli score precedenti, la risposta finale e gli eventi di tracciamento.
Il grafo LangGraph parte dal nodo `resolve_context`. Da qui il controllo passa al `Planner`; se il planner produce almeno uno step, il flusso prosegue verso l'`Executor`, altrimenti raggiunge direttamente il `Synthesizer`. Dopo l'esecuzione degli strumenti, il

controllo passa sempre al `Reflector`. A questo punto entra in gioco un arco condizionale: se la valutazione è sufficiente, oppure se il sistema decide di interrompere i tentativi di miglioramento, il flusso va al `Synthesizer`; se invece la risposta non è ancora sufficiente ma il trend degli score mostra un miglioramento utile, il controllo passa al `Replanner`, che genera un nuovo piano e riporta il workflow verso l'`Executor`.

Context Resolver. Il primo nodo del grafo è il *Context Resolver*. Questo agente ha il compito di rendere autonoma la domanda corrente dell'utente quando essa dipende dalla cronologia della conversazione.
L'implementazione combina regole deterministiche e chiamata LLM. Prima di invocare il modello, il nodo controlla se la domanda sembra già autonoma oppure se contiene segnali tipici di follow-up, come riferimenti ellittici, anni isolati o espressioni come "anche", "invece", "quello", "precedente". Se la domanda non richiede risoluzione, viene mantenuta invariata. Se invece appare dipendente dal contesto, il nodo invia al modello la domanda corrente e una versione compatta della cronologia conversazionale. L'output è un JSON contenente la domanda risolta, un breve riassunto del contesto, un indicatore che segnala se si tratta di un follow-up e un valore di confidenza.
Il risultato viene conservato, in questo modo i nodi successivi non lavorano soltanto sulla formulazione originale dell'utente, ma su una richiesta più esplicita e stabile.

Planner. Il *Planner* è l'agente responsabile della costruzione del piano di esecuzione. Riceve la domanda originale, la domanda risolta dal Context Resolver, il riassunto conversazionale e la cronologia disponibile. Prima di generare il piano, applica un guardrail di dominio: richieste chiaramente estranee agli appalti pubblici, ai dati ANAC o alla normativa sui contratti pubblici vengono bloccate e non producono alcuna chiamata agli strumenti.
Quando la richiesta è nel dominio del progetto, il Planner invoca il modello linguistico con uno schema JSON vincolato. Il risultato atteso è una lista di oggetti `Step`. Ogni step contiene:

`id`, `tool`, `args`, `depends_on`, `purpose`

dove `id` identifica lo step, `tool` indica lo strumento da chiamare, `args` contiene gli argomenti, `depends_on` specifica eventuali dipendenze da step precedenti e `purpose` descrive lo scopo dell'operazione.
Gli strumenti che il Planner può selezionare sono `text_to_sql_query`, `graph_rag_query`, `list_presets` e `run_athena_query`. Il primo viene usato per interrogazioni sui dati strutturati delle gare; il secondo per domande normative e recupero di informazioni dal grafo; il terzo per consultare query SQL predefinite; il quarto per eseguire direttamente query Athena esplicite. Una domanda ibrida può quindi essere scomposta in più step: ad esempio, una parte normativa viene assegnata a GraphRAG e una parte quantitativa a Text-to-SQL.
Il Planner contiene anche alcune correzioni deterministiche per casi specifici, ad esempio richieste normative multi-articolo o con anni espliciti, così da evitare piani troppo generici quando la struttura della domanda è riconoscibile tramite pattern.

Executor e invocazione degli strumenti. L'*Executor* riceve il piano generato dal Planner e si occupa di eseguirlo rispettando le dipendenze dichiarate. Ogni step parte con stato `pending`; quando tutte le sue dipendenze sono completate, lo step diventa eseguibile. Se più step sono indipendenti, l'Executor li esegue in parallelo, riducendo il tempo complessivo della richiesta.
Prima di chiamare uno strumento, l'Executor risolve eventuali placeholder presenti negli argomenti. Un placeholder nella forma `$1`, ad esempio, viene sostituito con l'output prodotto dallo step con identificativo 1. Questo meccanismo permette di costruire piani composizionali, nei quali una seconda operazione può usare il risultato di una prima interrogazione.
Le chiamate non vengono inviate direttamente ai microservizi applicativi. L'Executor utilizza un `MCPClient`, che comunica con il ***MCP Gateway***. Il gateway espone in modo uniforme gli strumenti disponibili e nasconde i dettagli dei servizi sottostanti. In particolare, le interrogazioni sui dati strutturati vengono indirizzate verso il servizio Text-to-SQL e poi verso Athena, mentre le richieste normative vengono inoltrate al servizio GraphRAG, che interroga il grafo Neo4j e produce una risposta basata sul contesto recuperato.
Per ogni step, l'Executor aggiorna lo stato con il risultato prodotto. Gli output vengono salvati sia in `tool_outputs`, indicizzati per identificativo dello step, sia nello `scratchpad`, che mantiene la sequenza cronologica delle evidenze raccolte. Se una chiamata fallisce, lo step viene marcato come `failed`; gli step che dipendono da esso vengono a loro volta bloccati o falliti. Ogni avvio, conclusione, risultato o errore viene inoltre tracciato tramite eventi.

Reflector e valutazione critica. Dopo l'esecuzione del piano, il workflow passa al *Reflector*. Questo agente svolge il ruolo di revisore critico dei risultati ottenuti. Riceve la domanda originale, la domanda risolta, il piano eseguito e tutte le evidenze raccolte nello scratchpad in ordine cronologico. Il suo compito non è produrre la risposta finale, ma stabilire se le informazioni raccolte siano sufficienti per rispondere correttamente all'utente.
Il Reflector invoca il modello linguistico con uno schema JSON che richiede tre campi:

`score`, `sufficient`, `feedback`

Lo `score` è un valore compreso tra 0 e 1; `sufficient` indica se il risultato è considerato adeguato; `feedback` descrive cosa manca o cosa dovrebbe essere migliorato. La soglia di sufficienza configurata nel progetto è pari a `0.8`. Se lo score è maggiore o uguale a tale soglia, il workflow può terminare la fase iterativa e passare alla sintesi.
Il Reflector è vincolato a valutare soltanto le evidenze effettivamente restituite dagli strumenti, il sistema non deve introdurre conoscenza esterna.
Ogni valutazione prodotta dal Reflector viene registrata nello stato. In particolare, lo score corrente viene salvato in `reflection_score`, il feedback in `reflection_feedback`, il numero di cicli viene incrementato in `reflection_count` e lo score viene aggiunto alla lista `reflection_scores`. Quest'ultima lista è fondamentale per la logica di ripianificazione adattiva.

Trend di ripianificazione. La decisione successiva al Reflector è implementata nella funzione di routing condizionale del grafo. Il sistema non decide di ripianificare soltanto perché lo score è inferiore a `0.8`; valuta anche se i tentativi precedenti stanno migliorando la qualità della risposta. Questo evita sia di fermarsi troppo presto, sia di entrare in un

ciclo di ripianificazioni inutili.
Il primo controllo è il limite massimo di cicli di reflection:

$$\texttt{ORCHESTRATOR_MAX_REFLECTION_CYCLES} = 4$$

Se il numero massimo di valutazioni critiche è stato raggiunto, il sistema passa comunque al Synthesizer. Successivamente viene controllata la soglia di sufficienza:

$$score \geq 0.8$$

Se la condizione è vera, il flusso segue la freccia verde del diagramma e va direttamente al *Synthesizer*. In questo caso la sintesi avviene perché la risposta è considerata sufficientemente completa.
Se invece lo score è inferiore alla soglia, il sistema concede sempre almeno un primo tentativo di ripianificazione. Dal secondo ciclo in poi, viene calcolata la variazione tra l'ultimo punteggio e quello precedente:

$$\Delta_i = score_i - score_{i-1}$$

Un miglioramento è considerato significativo se:

$$\Delta_i \geq 0.05$$

Se l'ultimo replan ha prodotto un incremento almeno pari a questa soglia, il sistema ritiene che la ripianificazione stia andando nella direzione corretta e autorizza un nuovo passaggio al Replanner.
Quando l'ultimo incremento non è sufficiente, il grafo analizza **due** ulteriori condizioni. La prima riguarda la stagnazione: vengono contate le transizioni consecutive in cui lo score non migliora in modo significativo o peggiora. Se la stagnazione supera il limite configurato, il ciclo viene interrotto. La seconda riguarda il trend recente: il sistema considera una finestra degli ultimi delta, configurata da `ORCHESTRATOR_TREND_WINDOW`, pari a 3, e verifica se al suo interno esiste almeno un miglioramento significativo. Questa condizione viene combinata con il guadagno complessivo:

$$score_{ultimo} - score_{iniziale} \geq 0.12$$

Se entrambe le condizioni sono vere, il sistema **concede un ulteriore replan** anche se l'ultimo passo non ha migliorato abbastanza. In altre parole, viene considerata non solo la variazione puntuale, ma anche la **traiettoria generale della qualità**.
Le frecce del diagramma rappresentano quindi tre casi distinti. La freccia verde indica che lo score è sufficiente e il sistema passa alla sintesi. La freccia arancione indica che lo score è insufficiente, ma il trend è positivo: il workflow passa al Replanner. La freccia rossa indica invece stagnazione o peggioramento: il sistema interrompe il ciclo e passa al Synthesizer con le evidenze disponibili.

Replanner. Il *Replanner* viene attivato quando il Reflector giudica insufficiente il risultato e la logica di trend stabilisce che un nuovo tentativo può essere utile. Riceve la domanda originale, la domanda risolta, il piano corrente, gli output già prodotti e il feedback critico. A partire da questi elementi, invoca il modello linguistico e genera un nuovo piano JSON.
Il nuovo piano può modificare gli argomenti degli step, aggiungere interrogazioni diagnostiche, rimuovere passaggi inutili oppure formulare in modo più preciso una richiesta verso lo stesso strumento. Ad esempio, nel caso di una risposta normativa incompleta, il Replanner può riprovare con una query GraphRAG più esplicita, includendo articolo, comma, decreto e anno di riferimento. Nel caso di una query sui dati strutturati troppo restrittiva, può introdurre una ricerca più ampia o una verifica intermedia.
Un aspetto importante dell'implementazione è il riuso degli step già completati. Se nel nuovo piano uno step ha lo stesso identificativo, lo stesso strumento e gli stessi argomenti di uno step precedente già concluso con successo, il risultato viene conservato e lo step rimane marcato come `done`. In questo modo il sistema evita chiamate ridondanti e non perde evidenze valide. Inoltre, se il nuovo piano dipende da uno step precedente non più presente nella lista aggiornata, il risultato viene reinserito come dipendenza già completata. Questo garantisce continuità tra i cicli di pianificazione.

Synthesizer e risposta finale. Il *Synthesizer* è l'agente incaricato di produrre la risposta finale per l'utente. Riceve la domanda originale, la domanda risolta, il riassunto della conversazione, il piano finale e tutte le evidenze raccolte dagli strumenti. A differenza del Reflector, che valuta la qualità del materiale disponibile, il Synthesizer ha il compito di trasformare tale materiale in una risposta coerente, leggibile e orientata all'utente.
La generazione avviene in streaming: il nodo invoca il client LLM e pubblica progressivamente i frammenti della risposta tramite eventi `final_answer_chunk`. Quando la generazione termina, il testo completo viene salvato in `final_answer` e viene emesso un evento finale. Il prompt del Synthesizer richiede di basare la risposta sulle evidenze disponibili, di non inventare informazioni non supportate e di preservare i riferimenti normativi o tabellari quando presenti.
Il Synthesizer può essere raggiunto per motivi diversi. Nel caso migliore, vi arriva perché il Reflector ha assegnato uno score sufficiente. In altri casi, vi arriva perché il piano è vuoto, perché la richiesta è fuori dominio o troppo generica, oppure perché il processo di ripianificazione non sta più producendo miglioramenti. In quest'ultimo scenario, la risposta finale deve riflettere i limiti delle evidenze disponibili, evitando conclusioni più forti di quanto i tool permettano di sostenere.

Streaming, osservabilità e tracciabilità. Durante l'intero ciclo di vita della richiesta, l'orchestratore produce eventi osservabili. Gli eventi principali includono la risoluzione del contesto, la creazione del piano, l'avvio di una chiamata tool, la conclusione della chiamata, il risultato ottenuto, eventuali errori, la reflection, la ripianificazione e la risposta finale. Questi eventi vengono pubblicati verso il frontend tramite Server-Sent Events, consentendo all'interfaccia di mostrare lo stato di avanzamento della richiesta in tempo reale.
Oltre allo streaming verso l'utente, gli eventi vengono conservati come `TraceEvent`. Ogni evento contiene un identificativo, un timestamp, il tipo di evento, il nodo che lo ha generato e un payload con i dettagli rilevanti. Questa tracciabilità permette di ricostruire l'intero percorso seguito dall'orchestratore: quale piano è stato generato, quali strumenti

sono stati chiamati, quali risultati sono stati ottenuti, quale score ha assegnato il Reflector e perché il sistema ha deciso di sintetizzare o ripianificare.
In conclusione, l'orchestratore multi-agente implementa un ciclo controllato di tipo *plan-execute-reflect-replan-synthesize*. La presenza di ruoli separati rende il sistema più modulare e ispezionabile; l'uso di `AgentState` consente di condividere informazioni tra i nodi; il MCP Gateway uniforma l'accesso agli strumenti; il Reflector introduce una valutazione critica della qualità; e la logica di trend evita ripianificazioni cieche, permettendo al sistema di continuare solo quando i punteggi indicano un miglioramento effettivo.

4.5 Gateway MCP

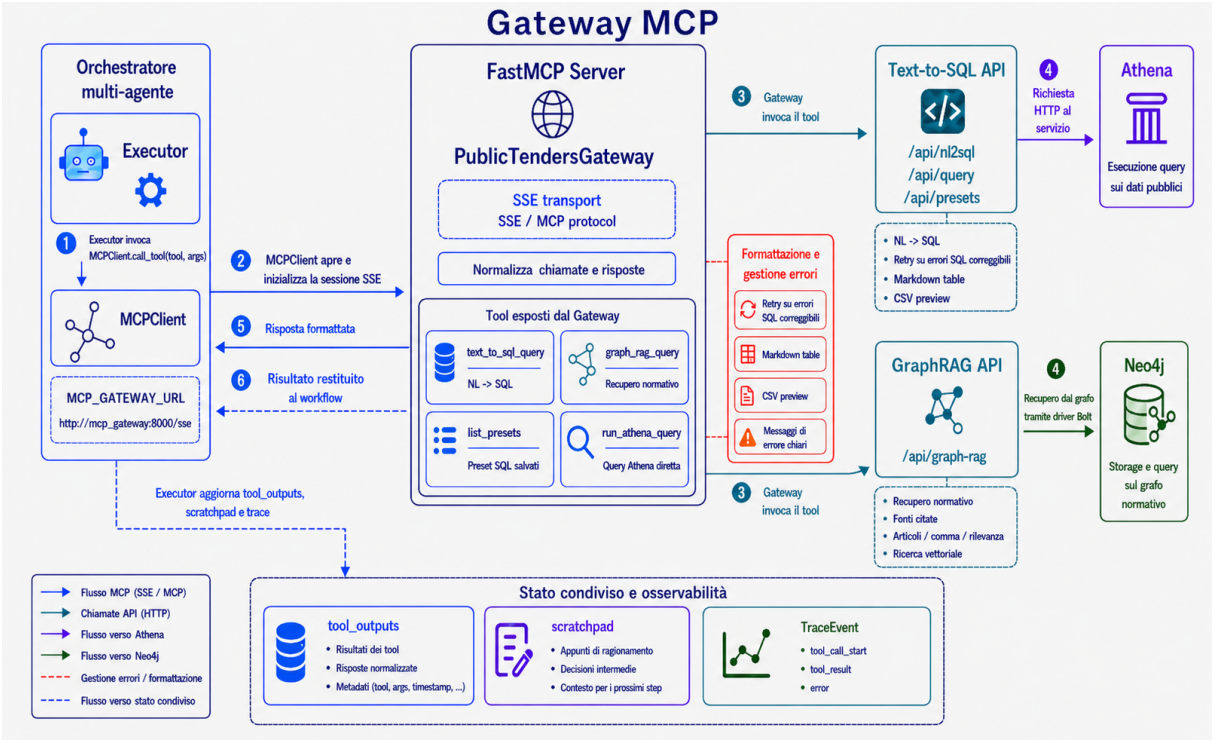


Figura 7: Architettura Gateway MCP

Il ***Gateway MCP*** costituisce il livello di intermediazione tra l'orchestratore multi-agente e i servizi applicativi specializzati. Nel progetto, infatti, l'Executor non invoca direttamente il servizio Text-to-SQL o il servizio GraphRAG, ma comunica con un unico punto di accesso: il **server MCP**. Questo componente implementa il protocollo MCP tramite `FastMCP` e registra un insieme di strumenti richiamabili dall'orchestratore attraverso una connessione Server-Sent Events.
L'obiettivo del Gateway MCP è uniformare l'accesso ai servizi eterogenei del sistema. Dal punto di vista dell'orchestratore, ogni operazione viene rappresentata come una chiamata a un tool con un nome e un insieme di argomenti. Dal punto di vista dei microservizi, invece, il gateway traduce tale chiamata in richieste HTTP verso endpoint FastAPI specifici. In questo modo il Planner può generare step astratti, come `text_to_sql_query` o `graph_rag_query`, senza dover conoscere gli endpoint, il formato interno delle API o i dettagli infrastrutturali dei servizi sottostanti.

Posizione del Gateway nell'architettura. Il Gateway MCP si colloca tra l'orchestratore e i servizi di dominio. L'orchestratore espone il workflow multi-agente, mentre il gateway espone gli strumenti operativi che permettono di interrogare i dati ANAC e il corpus normativo. Nel file `docker-compose.yaml`, il servizio `mcp_gateway` viene avviato sulla porta `8000` ed espone l'endpoint SSE usato dall'orchestratore.
Nel deployment Docker, il collegamento tra i servizi è configurato tramite variabili d'ambiente:

- `MCP_GATEWAY_URL`: endpoint SSE del Gateway MCP usato dall'orchestratore;
- `TEXT_TO_SQL_URL`: endpoint interno del servizio Text-to-SQL;
- `GRAPH_RAG_URL`: endpoint interno del servizio GraphRAG.

In particolare, l'orchestratore comunica con il gateway tramite l'endpoint SSE del servizio `mcp_gateway`, mentre il gateway inoltra le richieste ai servizi `text_to_sql` e `graph_rag`. Questa separazione consente di mantenere l'orchestratore indipendente dalla topologia dei microservizi: se un endpoint interno cambia, oppure se un servizio viene sostituito, è sufficiente aggiornare la configurazione o l'implementazione del gateway, senza modificare la logica del workflow LangGraph.

Comunicazione tra Executor e Gateway. Quando l'Executor deve eseguire uno step del piano, crea un'istanza di `MCPClient`. Il client legge l'URL SSE dalla configurazione e apre una sessione verso il Gateway MCP usando il trasporto `sse`. Una volta inizializzata la sessione, invoca il tool richiesto tramite `session.call_tool(tool_name, arguments)`. Il risultato viene restituito come contenuto testuale, ricomposto dal client e consegnato all'Executor.
L'Executor salva poi tale risultato nello stato condiviso dell'orchestratore: lo inserisce nel campo `result` dello step, nel dizionario `tool_outputs` indicizzato per identificativo dello step, e nello `scratchpad`, che conserva le evidenze in ordine cronologico. In parallelo vengono prodotti eventi di tracciamento come `tool_call_start`, `tool_call_end`, `tool_result` o `error`. In questo modo la chiamata MCP non è un'operazione isolata, ma entra nel ciclo di osservabilità dell'orchestratore.
Il client MCP incorpora inoltre una logica di robustezza. Le chiamate ai tool sono protette da timeout e gestione degli errori tramite la funzione `with_retry_and_timeout`. Nel caso specifico della chiamata a un tool, il timeout è impostato a `300` secondi, così da lasciare tempo sufficiente ai servizi downstream, che possono a loro volta eseguire query o recuperi semantici costosi.

Tool esposti dal Gateway. Il Gateway MCP registra quattro strumenti principali, ciascuno decorato con `@mcp.tool()`. Essi rappresentano l'interfaccia operativa che il Planner può inserire negli step del piano.

- `text_to_sql_query`: traduce una richiesta in linguaggio naturale in SQL e ne esegue la query sul data lake ANAC tramite Athena.
- `graph_rag_query`: inoltra una richiesta normativa al servizio GraphRAG, che recupera informazioni dal grafo Neo4j e restituisce una risposta con fonti normative.
- `list_presets`: restituisce l'elenco delle query SQL predefinite salvate nel sistema.

- `run_athena_query`: esegue direttamente una query SQL esplicita sul data lake ANAC.

Flusso del tool Text-to-SQL. Il tool `text_to_sql_query` è il più articolato tra quelli esposti dal gateway. Riceve una richiesta in linguaggio naturale e la inoltra al servizio Text-to-SQL tramite l'endpoint `/api/nl2sql`. Tale endpoint restituisce una query SQL generata dal modello del servizio strutturato. Successivamente il Gateway invia la query prodotta all'endpoint `/api/query`, che la esegue su Athena.
Il gateway applica anche alcune logiche di controllo prima e dopo l'esecuzione. In primo luogo, prova a inferire dalla richiesta dell'utente un limite esplicito sul numero di righe richieste. Se il limite non è specificato, il gateway usa una politica di anteprima: mostra un numero ridotto di righe nella risposta testuale, ma può includere un blocco CSV scaricabile con un numero più ampio di risultati. In secondo luogo, se Athena restituisce errori correggibili, come colonna non trovata, tabella non trovata o errore di sintassi SQL, il gateway effettua un secondo tentativo: reinvia al servizio Text-to-SQL la richiesta originale insieme alla query fallita e al messaggio di errore, chiedendo una correzione coerente con lo schema disponibile.
Il risultato finale viene formattato in Markdown. La risposta contiene la query SQL generata, il modello utilizzato dal servizio Text-to-SQL, un eventuale contesto semantico sulle colonne restituite e una tabella con i risultati dell'esecuzione. Questo formato è pensato per essere consumato dai nodi successivi dell'orchestratore, in particolare Reflector e Synthesizer.

Flusso del tool GraphRAG. Il tool `graph_rag_query` gestisce le richieste normative. Riceve una query testuale e la inoltra all'endpoint `/api/graph-rag` del servizio GraphRAG. Il payload contiene la domanda e il parametro `top_k`, che controlla il numero di elementi rilevanti da recuperare.
Il servizio GraphRAG interroga il corpus normativo rappresentato nel grafo Neo4j e produce una risposta testuale. Il Gateway MCP non si limita a inoltrare l'answer: estrae anche le fonti presenti nel risultato e le formatta in una sezione *Fonti normative citate*. Per ciascuna fonte vengono riportati il titolo della norma, il numero dell'articolo, l'eventuale titolo dell'articolo, il comma e il punteggio di rilevanza. Questa formattazione è importante perché permette al Synthesizer di produrre risposte normative più trasparenti e al Reflector di valutare se la risposta è supportata da fonti adeguate.

Preset SQL e query Athena dirette. Il tool `list_presets` consente di consultare le query SQL predefinite salvate nel sistema. Il Gateway invoca l'endpoint `/api/presets` del servizio Text-to-SQL e restituisce l'elenco dei preset disponibili con identificativo, titolo e query SQL. Questo strumento non esegue query: serve soltanto per mostrare all'utente o al workflow quali interrogazioni preconfigurate sono presenti.
Il tool `run_athena_query`, invece, riceve una query SQL esplicita e la inoltra direttamente all'endpoint `/api/query`. Nel progetto questo tool è pensato per casi in cui la query SQL è già nota o è stata prodotta esplicitamente dal workflow.

Gestione degli errori e normalizzazione dell'output. Un ruolo rilevante del Gateway MCP è la normalizzazione degli errori e delle risposte. Le chiamate HTTP verso i servizi downstream sono incapsulate in due funzioni helper:

- `fetch_json_post`, per le richieste HTTP di tipo POST;
- `fetch_json_get`, per le richieste HTTP di tipo GET.

In caso di errore HTTP, il gateway prova a estrarre il campo `detail` dalla risposta del servizio downstream e lo converte in un'eccezione leggibile. In caso di errore di connessione o di timeout, invece, produce un messaggio uniforme che segnala l'impossibilità di contattare il servizio downstream.
Questa normalizzazione evita che l'orchestratore debba conoscere le differenze tra gli errori prodotti dai vari microservizi. L'Executor riceve un errore testuale o un risultato testuale e aggiorna lo stato dello step di conseguenza. Successivamente, il Reflector può valutare se l'errore rende la risposta insufficiente e, se il trend lo consente, attivare il Replanner.

Vantaggi architetturali. L'introduzione del Gateway MCP offre diversi vantaggi. Innanzitutto, disaccoppia l'orchestratore dai servizi specialistici: l'orchestratore ragiona in termini di tool, non di endpoint HTTP. Inoltre, centralizza la formattazione dei risultati e la gestione degli errori, rendendo più uniforme il materiale passato al Reflector e al Synthesizer. Infine, rende il sistema estendibile: per aggiungere un nuovo servizio è sufficiente registrare un nuovo tool MCP e aggiornare il prompt del Planner, senza cambiare la struttura generale del workflow.
In sintesi, il Gateway MCP svolge la funzione di adattatore tra il livello agentico e il livello dei servizi. Da un lato espone strumenti semplici e richiamabili dall'orchestratore; dall'altro orchestra chiamate HTTP verso Text-to-SQL, Athena e GraphRAG, applicando logiche di validazione, retry, formattazione e normalizzazione dell'output.

4.6 Interfaccia utente Streamlit



Figura 8: Interfaccia GUI Streamlit

L'interfaccia utente di ANAC Query Assistant è realizzata mediante *Streamlit* e costituisce il punto di accesso principale alle funzionalità del sistema. Essa permette all'utente di formulare domande in linguaggio naturale, consultare le conversazioni precedenti e osservare in tempo reale le operazioni eseguite dall'orchestratore multi-agente.
L'interfaccia comunica esclusivamente con il servizio Orchestrator tramite API HTTP. In questo modo il frontend rimane indipendente dai servizi Text-to-SQL, GraphRAG, Neo4j e Amazon Athena, che vengono raggiunti indirettamente attraverso l'orchestratore e il Gateway MCP. Tale separazione consente di modificare i servizi interni senza intervenire sulla logica dell'interfaccia.

Visualizzazione dell'esecuzione. Le richieste vengono inviate all'endpoint di streaming `/agent/stream`. La comunicazione utilizza i *Server-Sent Events* (SSE), che permettono al frontend di ricevere progressivamente gli eventi generati dal workflow. Tra gli eventi visualizzati sono presenti:

- `plan_created`, contenente il piano prodotto dal Planner;
- `tool_call_start`, relativo all'avvio di uno strumento;
- `tool_result`, contenente il risultato restituito dal tool;
- `reflection`, con il punteggio e il feedback del Reflector;
- `replan`, generato in seguito a una ripianificazione;
- `final_answer`, contenente la risposta conclusiva.

Gli eventi vengono mostrati all'interno di un pannello di stato aggiornato durante l'esecuzione. L'utente può quindi osservare il piano iniziale, gli strumenti invocati, i relativi risultati, la valutazione del Reflector e le eventuali ripianificazioni. Al termine dell'elaborazione, queste informazioni rimangono consultabili nella sezione espandibile *Dettagli del Ragionamento & Esecuzione Tool*.

Conservazione delle conversazioni. La persistenza delle conversazioni non è affidata direttamente a Streamlit. Il frontend utilizza `st.session_state` esclusivamente per conservare lo stato temporaneo dell'interfaccia, come la conversazione selezionata, l'esecuzione attiva e i messaggi da visualizzare. La memorizzazione permanente è invece gestita dal servizio Orchestrator attraverso un database **SQLite** denominato `sessions.db`, collocato nella directory `services/orchestrator/data/`.
Il database contiene una tabella `sessions`, nella quale sono registrati l'identificativo, il titolo e la data di creazione di ogni conversazione, e una tabella `messages`, che conserva i messaggi dell'utente e dell'assistente. Per ciascun messaggio vengono memorizzati il ruolo, il contenuto, la data di creazione e i dettagli relativi al ragionamento e all'esecuzione degli strumenti. I messaggi sono collegati alla rispettiva sessione mediante una chiave esterna; l'eliminazione di una conversazione determina quindi anche la rimozione dei messaggi associati.

Visualizzazione e download dei risultati. Quando un tool restituisce dati tabellari, l'interfaccia produce un'anteprima direttamente nella risposta. Per risultati numerosi, la quantità di righe mostrata nella chat viene limitata, mentre il dataset completo può essere scaricato in formato CSV. È inoltre possibile esportare una conversazione come archivio di immagini, funzionalità utile per documentare i risultati e inserirli all'interno di relazioni o presentazioni.

4.7 Gestione delle configurazioni e dei container

La configurazione e l'avvio dei componenti di ANAC Query Assistant sono gestiti attraverso *Docker Compose*. Il backend è suddiviso in più servizi indipendenti, ciascuno eseguito all'interno di un container dedicato. Questa scelta permette di isolare le dipendenze, mantenere separati i diversi domini applicativi e riprodurre lo stesso ambiente di esecuzione su macchine differenti.
Il file `docker-compose.yaml` descrive cinque servizi principali: `neo4j`, `text_to_sql`, `graph_rag`, `mcp_gateway` e `orchestrator`. L'interfaccia Streamlit viene invece avviata separatamente nell'ambiente locale.

Configurazione centralizzata. Le configurazioni applicative sono definite mediante variabili d'ambiente, raccolte principalmente nel file `.env` situato nella directory principale del progetto. Tale file contiene le informazioni che possono cambiare tra un ambiente e l'altro, tra cui:

- credenziali per l'accesso ai servizi AWS;
- regione, database e posizione dei risultati di Amazon Athena;
- credenziali e database di Neo4j;
- chiavi API e modelli utilizzati dai provider LLM;
- provider e modello impiegati per la generazione degli embedding;
- parametri del Reflector e del meccanismo di ripianificazione;
- timeout delle comunicazioni tra Orchestrator, Gateway MCP e servizi downstream.

Immagine applicativa comune. I servizi applicativi vengono costruiti a partire da un unico `Dockerfile`, basato sull'immagine `python:3.10-slim`. Durante la fase di build vengono installate le dipendenze di sistema, copiate le dipendenze Python definite in `requirements.txt` e infine copiato il codice sorgente del progetto.
I container condividono quindi la stessa immagine di base, ma eseguono comandi differenti definiti nel file Compose. In particolare, Text-to-SQL, GraphRAG e Orchestrator avviano server ASGI mediante `uvicorn`, mentre il Gateway MCP viene eseguito come modulo Python. Questa soluzione mantiene uniforme l'ambiente software e riduce le differenze tra i servizi durante lo sviluppo.

Rete interna e accesso dal frontend. Docker Compose crea automaticamente una rete privata condivisa dai container. All'interno di questa rete, i servizi comunicano utilizzando nomi come `neo4j`, `text_to_sql`, `graph_rag` e `mcp_gateway`. L'uso di questi nomi rende la configurazione indipendente dagli indirizzi IP, che possono cambiare a ogni riavvio.
Il frontend non deve conoscere gli indirizzi di Text-to-SQL, GraphRAG, Neo4j o del Gateway MCP, poiché tutte le richieste vengono inoltrate attraverso l'Orchestrator.
Il flusso di comunicazione risultante è il seguente:

$$\texttt{Streamlit} \longrightarrow \texttt{Orchestrator} \longrightarrow \texttt{Gateway MCP}$$

$$\texttt{Gateway MCP} \longrightarrow \{\texttt{Text-to-SQL}, \texttt{GraphRAG}\}$$

$$\{\texttt{Text-to-SQL}, \texttt{GraphRAG}\} \longrightarrow \{\texttt{Athena}, \texttt{Neo4j}\}.$$

Persistenza e ricreazione dei container. La separazione tra container e volumi permette di aggiornare il codice senza perdere i dati applicativi. La rimozione o la ricreazione di un container non elimina automaticamente i volumi `neo4j_data`, `neo4j_logs` e `huggingface_cache`, né i file presenti nella directory montata dall'Orchestrator.
Quando viene modificato il file `.env`, i container devono essere ricreati, poiché le variabili d'ambiente vengono acquisite al momento della creazione e non vengono aggiornate dinamicamente. L'operazione può essere eseguita mediante:

```
docker compose up -d --build --force-recreate
```

Nel complesso, questa organizzazione separa le configurazioni dal codice, isola i componenti applicativi, rende esplicite le dipendenze e garantisce la persistenza dei dati necessari. L'architettura può inoltre essere estesa aggiungendo nuovi servizi o tool senza modificare il meccanismo generale di deployment.

4.8 Considerazioni implementative

Le scelte implementative adottate sono coerenti con gli obiettivi del progetto. FastAPI fornisce servizi leggeri e facilmente interrogabili; LangGraph permette di modellare il comportamento multi-agente come flusso controllato; FastMCP standardizza l'accesso ai tool; Neo4j rappresenta in modo naturale la conoscenza normativa; Athena e Glue permettono di interrogare in modo scalabile i dati strutturati ANAC.
Nel complesso, l'implementazione mira a mantenere separati i compiti dei singoli moduli, ma a integrarli nella fase in cui l'utente formula una domanda. Il valore del sistema non deriva da un singolo componente, ma dalla capacità dell'orchestratore di combinarli in modo dinamico, scegliendo il percorso più adatto al tipo di richiesta.

5 Valutazione sperimentale

La valutazione sperimentale ha l'obiettivo di verificare non soltanto il funzionamento dei singoli componenti, ma anche la capacità complessiva di ANAC Query Assistant di interpretare la domanda dell'utente, selezionare gli strumenti appropriati e produrre risposte pertinenti, corrette e complete. Particolare attenzione è dedicata alle richieste ibride, che richiedono l'integrazione tra dati strutturati provenienti da Amazon Athena e contenuti normativi recuperati mediante Neo4j GraphRAG.

5.1 Obiettivi della valutazione

La sperimentazione è stata organizzata attorno alle seguenti domande di ricerca:

- **RQ1**: l'orchestratore seleziona correttamente gli strumenti necessari rispetto alla tipologia della richiesta?
- **RQ2**: le risposte generate risultano coerenti, corrette e complete rispetto alla risposta di riferimento?
- **RQ3**: il meccanismo composto da Reflector e Replanner produce un miglioramento osservabile della risposta?
- **RQ4**: il sistema gestisce correttamente domande ambigue o incomplete, evitando l'esecuzione non necessaria degli strumenti?
- **RQ5**: quali sono i tempi di elaborazione delle diverse categorie di richieste?

5.2 Dataset di valutazione

Il dataset sperimentale contiene 20 domande formulate in linguaggio naturale, suddivise equamente in quattro categorie.

Categoria	**Descrizione**	**Casi**
SQL-only	Domande quantitative che richiedono esclusivamente l'interrogazione dei dati strutturati mediante Text-to-SQL e Amazon Athena.	5
RAG-only	Domande giuridiche che richiedono il recupero di articoli, commi e riferimenti normativi mediante Neo4j GraphRAG.	5
Hybrid	Domande che combinano dati quantitativi e informazioni normative, richiedendo l'esecuzione di entrambi gli strumenti.	5
Ambiguous	Domande generiche o incomplete per le quali il sistema dovrebbe richiedere un chiarimento senza eseguire strumenti.	5
Totale		**20**

Tabella 4: Composizione del dataset di valutazione

Per ciascun caso sono specificati la domanda, la categoria, l'insieme degli strumenti attesi e una descrizione concettuale della risposta di riferimento. L'insieme comprende richieste puramente quantitative, interrogazioni normative, domande ibride e casi nei quali il guardrail dovrebbe impedire l'esecuzione di strumenti non necessari.

5.3 Configurazione e protocollo sperimentale

Ogni domanda è stata inviata direttamente al workflow LangGraph dell'orchestratore, inizializzando un nuovo `AgentState` privo di cronologia conversazionale. Il frontend Streamlit non è stato coinvolto nella misurazione; la latenza osservata rappresenta pertanto il tempo impiegato dal workflow per produrre la risposta finale e non comprende la successiva valutazione del giudice esterno.
Il sistema sottoposto a valutazione utilizza `gemini-3.1-pro-preview`, mentre la valutazione automatica delle risposte è affidata a un modello distinto, `gemini-2.5-pro`. Questa separazione riduce il rischio che lo stesso modello valuti direttamente le proprie risposte. Per ogni caso sono stati registrati:

- il piano generato dal Planner;
- i tool effettivamente avviati durante tutti i cicli di esecuzione;
- la risposta finale;
- i punteggi di correttezza e completezza del giudice esterno;
- i punteggi prodotti dal Reflector;
- il numero di cicli di riflessione e ripianificazione;
- la latenza complessiva del workflow;
- gli eventuali errori di esecuzione.

I risultati riportati si riferiscono a un'esecuzione completa del dataset successiva alla correzione della metrica di selezione degli strumenti. Tale metrica considera tutti i tool realmente avviati e penalizza sia gli strumenti mancanti sia quelli eseguiti inutilmente.

5.4 Metriche

L'accuratezza nella selezione degli strumenti è definita come:

$$A_{\text{tool}} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}\left(T_i = \widehat{T}_i\right), \tag{1}$$

dove T_i rappresenta l'insieme degli strumenti attesi, $\widehat{T}_i$ l'insieme degli strumenti effettivamente eseguiti e $\mathbb{I}$ la funzione indicatrice. Il confronto tra i due insiemi è esatto.
La qualità della risposta viene valutata mediante due punteggi compresi tra 0 e 1:

- **correctness**, relativo alla correttezza e alla pertinenza delle informazioni fornite;
- **completeness**, relativo alla copertura di tutti gli aspetti richiesti dalla domanda.

Il *tasso di superamento della risposta* considera positivi i casi con correctness maggiore o uguale a 0,8. È stata inoltre introdotta una metrica più restrittiva di successo end-to-end:

$$S_i = \mathbb{I}\left(T_i = \widehat{T}_i \wedge \mathit{correctness}_i \geq 0.8 \wedge \mathit{completeness}_i \geq 0.8\right). \tag{2}$$

Una prova è quindi considerata **completamente riuscita** solo quando il sistema seleziona esattamente gli strumenti previsti e produce una risposta sufficientemente corretta e completa.
L'effetto di una ripianificazione viene misurato mediante:

$$\Delta_{\text{replan}} = score_{\text{finale}} - score_{\text{iniziale}}. \tag{3}$$

Per le prestazioni sono state calcolate latenza media, mediana e novantacinquesimo percentile, indicato come P_{95}.

5.5 Risultati

La Tabella 5 riporta i risultati aggregati per categoria.

Categoria	**Tool acc.**	**Correttezza**	**Completezza**	**Successo**	**Latenza**
SQL-only	100%	100%	100%	100%	31,85 s
RAG-only	100%	98%	92%	80%	49,02 s
Hybrid	100%	84%	100%	60%	71,02 s
Ambiguous	80%	100%	100%	80%	21,59 s
Complessivo	95%	95,5%	98%	80%	43,37 s

Tabella 5: Risultati della valutazione sperimentale

Il sistema ha selezionato esattamente gli strumenti attesi in 19 casi su 20, ottenendo un'accuratezza del 95%. La correttezza media assegnata dal giudice esterno è pari al 95,5%, mentre la completezza media raggiunge il 98%. Diciotto risposte su venti superano la soglia di correttezza, corrispondente a un *answer pass rate* del 90%. Applicando il criterio più restrittivo, che considera anche completezza e selezione degli strumenti, il successo end-to-end è pari all'80%, ossia 16 casi su 20.
La latenza media complessiva è pari a 43,37 secondi, con una mediana di 46,94 secondi e un P_{95} di circa 76,64 secondi. Le domande ambigue risultano mediamente più rapide, poiché nella maggior parte dei casi vengono interrotte dal guardrail senza eseguire strumenti. Le richieste ibride presentano invece la latenza media più elevata, pari a 71,02 secondi, poiché richiedono il recupero normativo, l'interrogazione dei dati strutturati e la successiva integrazione delle evidenze.
Le richieste SQL-only sono state completate correttamente in tutti i casi. Anche il percorso RAG-only presenta risultati elevati, sebbene una risposta sul subappalto sia risultata incompleta. La categoria Hybrid costituisce il principale punto critico: la selezione dei tool è sempre corretta, ma il tasso di successo end-to-end si ferma al 60%.

5.6 Comportamento del Reflector e del Replanner

Sedici richieste hanno determinato l'esecuzione di almeno uno strumento. Su questi casi sono stati registrati complessivamente 17 cicli di riflessione. Una sola richiesta ha attivato il Replanner.
Il caso riguardava il calcolo della somma degli importi aggiudicati in Lombardia nel 2024. La prima valutazione del Reflector ha prodotto un punteggio pari a 0,4. Dopo la ripianificazione, il nuovo risultato ha ottenuto un punteggio pari a 1,0:

$$\Delta_{\text{replan}} = 1.0 - 0.4 = 0.6. \tag{4}$$

La risposta finale è stata valutata positivamente anche dal giudice esterno. Il caso fornisce pertanto un'evidenza concreta dell'utilità del Replanner quando il Reflector individua correttamente un risultato insufficiente.
Sono tuttavia emersi alcuni problemi di calibrazione. In due richieste ibride il Reflector ha assegnato rispettivamente punteggi pari a 0,9 e 1,0, mentre il giudice esterno ha attribuito a entrambe una correttezza pari a 0,6. Poiché il punteggio interno aveva già superato la soglia di 0,8, il workflow ha terminato l'elaborazione senza attivare una nuova ripianificazione.
Un'ulteriore risposta RAG-only ha ottenuto un punteggio interno pari a 1,0, ma una completezza esterna pari a 0,6. Questi casi mostrano che il Replanner può intervenire soltanto quando il Reflector identifica correttamente le carenze della risposta. L'efficacia della ripianificazione dipende quindi anche dalla calibrazione del meccanismo di valutazione interno.

5.7 Analisi degli errori

L'analisi dei singoli casi ha evidenziato quattro situazioni principali.
La domanda relativa al subappalto nel nuovo Codice è stata gestita con il tool corretto, ma la risposta si è concentrata sul primo comma dell'articolo 119, senza descrivere in modo completo l'iter autorizzativo e gli ulteriori obblighi procedurali. La correttezza è risultata elevata, mentre la completezza si è fermata a 0,6.
Nella risposta sui termini di presentazione delle offerte è stata individuata una lieve imprecisione temporale relativa all'entrata in vigore e all'efficacia delle disposizioni del D.Lgs. 36/2023. Il giudice ha pertanto assegnato una correttezza pari a 0,9.
Due richieste ibride hanno ottenuto un punteggio di correttezza pari a 0,6. Nel primo caso, relativo alle gare aggiudicate a PMI, il dato numerico non era accompagnato da evidenze sufficienti a consentire al giudice esterno di verificarlo. Nel secondo caso, il numero di applicazioni del soccorso istruttorio è stato stimato ricercandone la menzione nei dati della gara. La presenza dell'espressione nell'oggetto o nella descrizione non costituisce tuttavia una prova certa della sua effettiva applicazione.
Infine, la domanda "Dimmi di più sul subappalto", classificata nel dataset come ambigua, ha determinato l'esecuzione di GraphRAG. La risposta prodotta è stata giudicata corretta e completa, ma la selezione dello strumento non corrispondeva alla ground truth, che prevedeva una richiesta di chiarimento. Il caso evidenzia un'ambiguità nell'etichettatura: la domanda può essere interpretata come follow-up privo di contesto oppure come richiesta normativa autonoma sul subappalto.
Non sono stati osservati errori tecnici, timeout o indisponibilità dei servizi durante l'esecuzione completa del benchmark.

5.8 Limiti della valutazione

La sperimentazione presenta alcuni limiti. Il dataset è composto da soli 20 casi e non copre tutte le possibili formulazioni linguistiche. I risultati si riferiscono inoltre a una singola esecuzione valida per ciascuna domanda. La natura non deterministica dei modelli rende opportuno ripetere il benchmark più volte e riportare media e deviazione standard.
Il giudice automatico riceve la domanda, la risposta finale e una descrizione concettuale del risultato atteso. Per alcune richieste quantitative tale descrizione non contiene il valore numerico esatto ottenuto mediante una query di controllo indipendente. Il punteggio

di correctness misura quindi principalmente la coerenza semantica con il riferimento e non costituisce, da solo, una certificazione della correttezza fattuale.
L'impiego di un LLM come giudice può inoltre introdurre bias e variabilità. Sebbene utilizzi un modello distinto da quello del sistema valutato, i risultati normativi e quantitativi più rilevanti devono essere verificati manualmente confrontandoli rispettivamente con i testi ufficiali e con query Athena predisposte come ground truth.

5.9 Considerazioni conclusive

La valutazione mostra che ANAC Query Assistant seleziona correttamente gli strumenti nella maggior parte dei casi e raggiunge risultati particolarmente elevati nelle richieste SQL-only. Le principali difficoltà emergono nelle domande ibride, nelle quali è necessario integrare correttamente evidenze normative e dati quantitativi.
Il caso di ripianificazione osservato conferma che il Replanner può migliorare in modo significativo un risultato inizialmente insufficiente. Le differenze tra i punteggi del Reflector e quelli del giudice esterno evidenziano tuttavia la necessità di migliorare la calibrazione della valutazione interna e di rafforzare le ground truth quantitative del benchmark.

6 Conclusioni e sviluppi futuri

Il lavoro svolto ha portato alla realizzazione di ANAC Query Assistant, un sistema conversazionale che semplifica l'accesso ai dati sugli appalti pubblici e alla normativa italiana. L'utente può formulare domande in linguaggio naturale senza conoscere SQL, Amazon Athena, Cypher o la struttura delle fonti interrogate.
Il sistema integra dati strutturati, relativi a gare, lotti, aggiudicazioni e stazioni appaltanti, con contenuti normativi recuperati mediante Neo4j GraphRAG. L'orchestratore multi-agente coordina la risoluzione del contesto, la pianificazione, l'esecuzione degli strumenti, la valutazione dei risultati, l'eventuale ripianificazione e la sintesi finale. Questa architettura consente di combinare evidenze quantitative e riferimenti normativi selezionati anche in funzione del periodo indicato dall'utente.

Limiti dell'implementazione attuale. L'architettura costituisce un prototipo funzionale eseguito tramite Docker Compose su un singolo host. Non sono presenti replica automatica, bilanciamento del carico o meccanismi di continuità operativa. La persistenza delle conversazioni mediante SQLite, la conservazione delle tracce in formato JSONL e l'utilizzo di code SSE in memoria risultano adeguati per lo sviluppo locale, ma non permettono uno scaling orizzontale sicuro.
La qualità delle risposte dipende inoltre dalla completezza e dall'aggiornamento delle fonti. Il ciclo di riflessione e ripianificazione migliora la capacità del sistema di raccogliere evidenze adeguate, ma non garantisce in modo assoluto la correttezza giuridica. Le risposte normative devono quindi essere considerate uno strumento di supporto alla consultazione.

6.1 Evoluzione verso un'architettura cloud AWS

Uno dei principali sviluppi futuri consiste nella migrazione del livello applicativo verso Amazon Web Services. Il progetto utilizza già Amazon S3, AWS Glue e Amazon Athena per la gestione e l'interrogazione dei dati ANAC; l'evoluzione riguarderebbe quindi soprattutto Orchestrator, Gateway MCP, Text-to-SQL e GraphRAG, attualmente eseguiti in locale.
Le immagini Docker potrebbero essere pubblicate su Amazon ECR e distribuite mediante Amazon ECS con AWS Fargate. La separazione tra i microservizi consentirebbe di scalare ciascun componente in modo indipendente, migliorando disponibilità e gestione del carico.

Esternalizzazione dello stato. Per supportare più repliche dell'Orchestrator, SQLite potrebbe essere sostituito da Amazon RDS, Aurora PostgreSQL o DynamoDB. Le tracce potrebbero essere archiviate in Amazon S3, mentre la distribuzione degli eventi SSE potrebbe essere supportata da Amazon ElastiCache for Redis. La persistenza dello stato di esecuzione permetterebbe inoltre di riprendere le richieste interrotte in seguito al riavvio o alla sostituzione di un container.

Evoluzione dei dati e del grafo normativo. Il data lake potrebbe essere organizzato nei livelli Bronze, Silver e Gold. Il livello Bronze conserverebbe i dati originali, il Silver conterrebbe dati normalizzati e validati, mentre il Gold offrirebbe aggregazioni ottimizzate per le interrogazioni più frequenti. Ciò ridurrebbe i dati letti da Athena e semplificherebbe le query generate dal servizio Text-to-SQL.

Neo4j potrebbe essere mantenuto su infrastruttura AWS oppure sostituito da una soluzione gestita, previa verifica della compatibilità con query Cypher, indici vettoriali, metadati temporali e procedure di ingestion.

Sicurezza e osservabilità. Nel deployment cloud, i container dovrebbero utilizzare ruoli IAM con privilegi minimi, evitando credenziali AWS statiche nei file di configurazione. Le chiavi dei provider LLM e le credenziali Neo4j potrebbero essere archiviate in AWS Secrets Manager e protette mediante AWS KMS.
Amazon CloudWatch consentirebbe di centralizzare log, metriche e allarmi, monitorando durata delle richieste, consumo di risorse, errori dei tool e prestazioni dei microservizi. L'integrazione con Amazon Bedrock potrebbe infine offrire un accesso gestito a diversi modelli generativi, da valutare in termini di qualità, latenza, costo e affidabilità degli output strutturati.

Strategia di migrazione. La migrazione potrebbe essere realizzata progressivamente attraverso:

1. pubblicazione delle immagini Docker in Amazon ECR;
2. deployment dei microservizi su Amazon ECS con AWS Fargate;
3. esternalizzazione di conversazioni, tracce, eventi e stato di esecuzione;
4. centralizzazione di log e metriche mediante CloudWatch;
5. gestione di credenziali e permessi mediante Secrets Manager e IAM;
6. consolidamento del data lake nei livelli Bronze, Silver e Gold;
7. valutazione di Amazon Bedrock e della soluzione cloud per Neo4j;
8. introduzione di autoscaling, test di carico e procedure di continuità operativa.

In conclusione, ANAC Query Assistant dimostra come un'architettura multi-agente possa coordinare fonti strutturate e normative attraverso strumenti specializzati e un processo di elaborazione osservabile. La migrazione verso AWS permetterebbe di trasformare il prototipo in un servizio condiviso, scalabile e monitorabile. Resterebbe comunque essenziale mantenere un processo continuo di aggiornamento delle fonti, valutazione delle risposte e verifica delle query generate.